Gerador de Missões Unooria

Guia de funcionamento: o que a esteira faz, por que faz assim,
e como escrever uma entrada que produz uma boa missão

Documentação técnica — repositório `gerador-missoes`

19 de setembro de 2026

Resumo

O gerador transforma uma ideia de missão em um **PRD pronto para codificação** e um **one-pager que o time pedagógico aprova**, passando por 50 agentes organizados em três etapas e seis ciclos de correção automática. Este guia é para quem vai *usar* a ferramenta: explica o que colocar na entrada, o que cada etapa faz, como ler o diagnóstico quando algo trava, e por que o sistema foi desenhado desta forma e não como um único prompt grande. A Seção 2 — com exemplos de briefing bom e ruim — é a que mais muda o resultado na prática.

Sumário

1 O que a esteira faz **2**
1.1 A regra que muda tudo: a esteira não desiste 2

2 Como escrever a entrada **2**
2.1 O que o primeiro agente precisa 2
2.2 Um briefing que funciona 3
2.3 Briefings que produzem missão ruim 3
2.4 Anexos valem mais que adjetivos 4

3 Por que 50 agentes, e não um prompt grande **4**
3.1 Densidade de instrução degrada o cumprimento 4
3.2 O critério de decomposição 4
3.3 Redundância não é cobertura 5
3.4 O contraponto honesto 5

4 As etapas **5**
4.1 Produto (01–15): a missão pode ser construída? 5
4.2 Design (16–31): como isso se constrói 6
4.3 Dev (32–48): isso é implementável? 6
4.4 Transversal (49–50): o que aconteceu 6

5 Os seis ciclos de correção **6**

6 Os portões, e o que fazer quando reprovam **7**

7 Como ler o diagnóstico **8**

8 O identificador da missão **8**

9 Configuração **8**

10 Três recomendações de uso **9**

1 O que a esteira faz

Você escreve um briefing de meia página sobre a missão que quer. A esteira devolve:

Artefato	Para quem é
`one-pager-pedagogico.md` (+ `.docx`)	Time pedagógico, coordenação e a professora consultora. Linguagem de sala de aula, zero jargão. É o documento de aprovação.
`PRD.md`	O agente (ou a pessoa) que vai codificar. 25 seções, com contratos de dados, eventos e arquitetura.
`diagnostico-execucao.md`	Você. Diz o que passou, o que passou com ressalva, o que travou e de quem é cada conserto.
`design/09-diagramas.html`	Quem quiser *ver* a missão: jornada e fluxo em diagramas.
`design/08-prototipo.html`	Revisão humana: as telas navegáveis, abre com duplo clique.
`aprendizado.md`	O processo: o que esta execução ensinou e o que propor de mudança nas rubricas.

Mais os documentos intermediários: spec da missão, jornada, fluxo, copy, especificação de telas, camada de acessibilidade, handoff de dev e os relatórios de cada portão.

1.1 A regra que muda tudo: a esteira não desiste

Um portão reprovado **não** interrompe a execução. Todos os documentos são produzidos até o fim; o que deriva de uma etapa reprovada sai marcado `[PROVISÓRIO]`, e o diagnóstico final diz o que é firme e o que é rascunho.

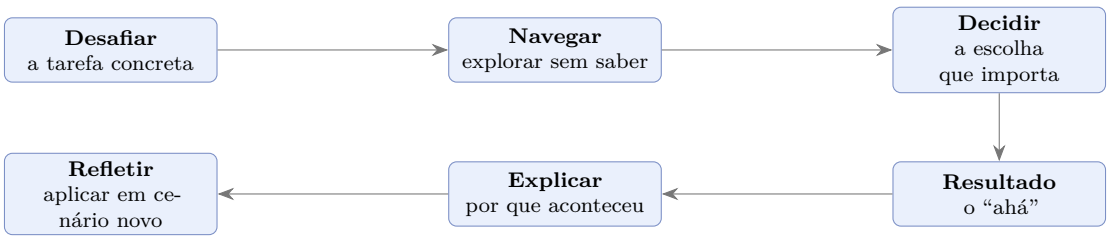
Isso é deliberado. Na versão anterior, a primeira reprovação parava tudo: não saía PRD, não saía one-pager, não saía relatório — e quem precisava entender o erro ficava sem material para analisar. Um sistema que falha sem deixar evidência transfere para a pessoa o trabalho de reconstruir o que aconteceu.

2 Como escrever a entrada

Esta é a seção com maior efeito sobre o resultado. A esteira é boa em *estruturar*, *auditar* e *formalizar*. Ela é ruim em adivinhar o que você não disse — e, por desenho, prefere marcar uma lacuna a inventar conteúdo.

2.1 O que o primeiro agente precisa

O agente 01 (Núcleo Narrativo) produz quatro coisas a partir do seu briefing: a **descoberta central**, a **narrativa**, o **mecanismo das seis etapas** e a **identificação** (ano, bimestre, ambiente). Ele precisa de matéria-prima para as quatro.



*O estudante que vive o Navegar **não pode saber** o que o Resultado vai revelar. Esse é o teste mais duro da esteira.*

Figura 1: O framework de seis etapas que toda missão segue

2.2 Um briefing que funciona

```
Missao para o 5o ano do Fundamental, 3o bimestre, no ambiente do YouTube (unitube).

Tema do bimestre (material Entre Redes anexado): como a recomendacao de
conteudo funciona e por que duas pessoas veem coisas diferentes.

O que quero que a crianca descubra: que o que ela ve no feed nao e "o que
existe", e sim o que o sistema aprendeu que ela costuma assistir.

Ideia de mecanismo: ela monta a playlist de um personagem a partir de um banco
de videos. Depois ve o feed que aquele personagem passa a receber. Dois colegas
com playlists diferentes recebem feeds diferentes, e isso fica visivel na tela.

Restricoes: 50 minutos, aula unica, dois estudantes por tablet.
A professora precisa conseguir pausar a turma antes da revelacao.

Em aberto: nao sei se o banco deve ter 8 ou 12 videos.
```

Por que funciona:

1. **Ano e bimestre explícitos.** O identificador da missão codifica os dois (Seção 8); sem eles o primeiro portão determinístico já reprova.
2. **A descoberta está escrita como descoberta**, não como objetivo de aula. Compare: "a criança vai aprender sobre algoritmos de recomendação" (objetivo) contra "o que ela vê não é o que existe" (descoberta).
3. **Há um mecanismo, não só uma intenção.** "Monta a playlist, vê o feed que resulta, compara com o colega" é uma cadeia de ações. "Trabalhar pensamento crítico sobre algoritmos" não é.
4. **As restrições de sala aparecem.** Tempo, dispositivo compartilhado e controle da professora mudam o desenho, e a esteira audita os três.
5. **A dúvida está declarada.** Dizer "não sei se 8 ou 12" faz a esteira decidir e registrar a decisão. Omitir faria ela decidir sem você saber.

2.3 Briefings que produzem missão ruim

Entrada problemática	O que acontece
"Missão sobre fake news para o Fundamental."	Sem ano, sem bimestre, sem ambiente, sem mecanismo. A esteira inventa os quatro, e o que ela inventa é o mais genérico possível. **Pior caso de uso da ferramenta.**
"O estudante navega por um feed com posts falsos e descobre que foram manipulados."	O briefing **entrega a virada**. A esteira tem um portão dedicado a isso (agente 02) e vai reprovar, mas o estrago já aconteceu: a ideia nasceu sem segredo, e refazer o núcleo custa rodadas.
"Uma missão que desenvolva pensamento crítico e cidadania digital."	Intenção sem mecanismo. O gate estrutural tem um critério só para isso: se você remover a frase, a missão continua igual, então a frase não descreve nada construível.
"O estudante avalia 15 perfis e classifica cada um como confiável ou não."	Passa no teste de decisão simples e reprova no volume: 15 decisões do mesmo tipo excede o teto da faixa. Vira **ressalva** ou reprovação no agente 09.

continuação

Entrada problemática	O que acontece
"Fazer no Roblox, que as crianças amam."	Ambiente fora da lista fechada (Instagram, YouTube, ChatGPT, TikTok, WhatsApp). Reprovação imediata — não existe seção de design system para construir.
"A criança conversa com a IA e reflete sobre o que aprendeu."	Reflexão genérica. O critério 1.6 pede uma *provocação concreta*: uma situação nova em que ela aplica o que viveu. O teste é direto: alguém que não fez a missão consegue responder?
Briefing de oito páginas com todas as telas já desenhadas.	A esteira vira transcritora. Os portões detectam "eco" — quando um agente repete a entrada em vez de construir — mas você terá pago 50 chamadas para formatar o que já tinha.

2.4 Anexos valem mais que adjetivos

Anexe o material do bimestre (o *Entre Redes* em `.docx`). Ele é a fonte primária de código curricular e de contexto: a esteira lê os códigos da BNCC dali e os confere contra o anexo oficial do MEC [2]. Dois parágrafos de material real valem mais que uma página de descrição.

3 Por que 50 agentes, e não um prompt grande

A pergunta aparece sempre, e a resposta não é "porque especialização é boa".

3.1 Densidade de instrução degrada o cumprimento

A etapa de Produto sozinha carrega cerca de 74 mil *tokens* de instrução e 51 critérios tabelados. No *benchmark* IFScale, que mede exatamente isso, os melhores modelos de fronteira chegam a 68% de acurácia quando precisam seguir 500 instruções ao mesmo tempo [4], com viés sistemático em favor das instruções apresentadas primeiro. Liu et al. [7] documentam o efeito irmão em contexto longo: o que está no meio é usado pior que o que está nas pontas.

Há evidência interna, e ela é mais convincente que a externa. O prompt do agente 09 abre justificando a própria existência: *"esta avaliação existe porque a rubrica original errava sistematicamente aqui: os critérios de faixa etária competiam por atenção com ∼30 critérios de outra natureza"*. A versão agregada falhava de modo reprodutível; a decomposta corrigiu.

3.2 O critério de decomposição

Não se divide por etapa cronológica. Divide-se por **decisão encapsulada**, no sentido clássico de Parnas [8]: cada agente responde uma pergunta que pode ser respondida — e errada — independentemente das outras, e emite um envelope JSON que um portão consegue ler.

Agente	A pergunta que ele responde
05 Gate estrutural	A missão tem estrutura construível?
08 Avaliação pedagógica	Cumpre currículo e protege o estudante?
09 Experiência do estudante	Alguém dessa idade consegue viver isso?
29 QA de entregáveis	A construção é fiel ao que foi especificado?

3.3 Redundância não é cobertura

Três critérios do agente 09 eram testes idênticos a três critérios do agente 05 — mesma pergunta, mesmo modo de falha. Foram removidos. Knight e Leveson [6] mostraram, em 27 implementações independentes submetidas a um milhão de testes, que falhas correlacionadas anulam o ganho da redundância. Quando os dois "avaliadores independentes" são prompts diferentes do *mesmo modelo*, sobre o *mesmo texto*, a correlação é ainda maior.

3.4 O contraponto honesto

A literatura recente é cética quanto a sistemas multiagente: sob orçamento fixo de *tokens*, um agente único frequentemente iguala ou supera arquiteturas multiagente [10]. A condição de contorno declarada pelos próprios autores é que o multiagente vence *"quando a utilização efetiva de contexto de um agente único está degradada"* — que é exatamente o regime desta esteira. E Cemri et al. [1], sobre mais de 1.600 execuções anotadas, mostram que decompor cria uma classe nova de falha: contratos de interface. Por isso cada fronteira aqui tem envelope JSON verificado por teste.

4 As etapas

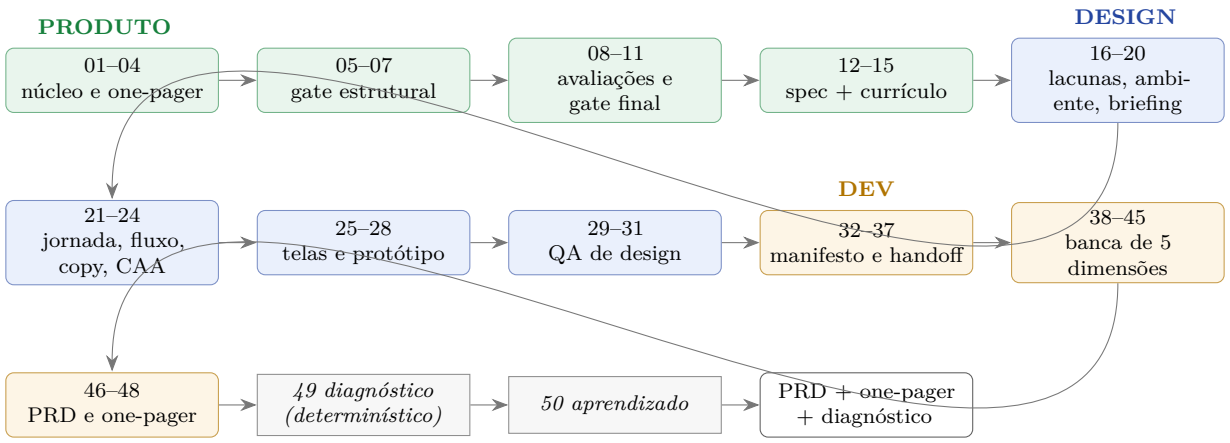


Figura 2: Os 50 agentes, numerados na ordem de execução

4.1 Produto (01–15): a missão pode ser construída?

Decide *o que* a missão é. Começa pelo núcleo narrativo, passa por um micro-portão que caça *spoiler* e ausência de escolha significativa, completa o one-pager de 18 seções, submete tudo aos 23 critérios do gate estrutural, avalia currículo e faixa etária em paralelo, e fecha com a **spec** — o contrato de 12 blocos contra o qual Design e Dev constroem.

4.2 Design (16–31): como isso se constrói

Fecha as lacunas da spec, decide o ambiente simulado, escreve o briefing de ofício, e então constrói: jornada e fluxo em **diagramas Mermaid**, copy em duas passadas, camada de acessibilidade, especificação de telas e um **protótipo HTML navegável** que abre com duplo clique. Fecha com o QA de entregáveis, que audita fidelidade à spec.

> **Nota sobre ferramentas.** A esteira não usa mais o Figma — não há cliente oficial que o sirva neste ambiente, e board em ferramenta que ninguém abre não é entregável. Jornada e fluxo saem em Mermaid, as telas saem no Stitch e como especificação em Markdown, e o artefato visual é o protótipo HTML.

4.3 Dev (32–48): isso é implementável?

Confere a ficha do manifesto, produz o handoff e o brief de assets, valida o contrato de telemetria, e submete tudo a uma banca de cinco dimensões em paralelo — produto, design, pedagogia, dados/LGPD e arquitetura. Consolida, escreve o PRD e, por último, o one-pager pedagógico.

4.4 Transversal (49–50): o que aconteceu

O diagnóstico determinístico monta o dossiê a partir dos envelopes que os portões gravaram; o agente de aprendizado escreve a leitura em prosa e registra o que a execução ensinou.

5 Os seis ciclos de correção

Quando um portão reprova, quem decide repetir é o **orquestrador**, em código — não uma instrução em linguagem natural dentro do prompt, que o modelo pode ou não seguir.

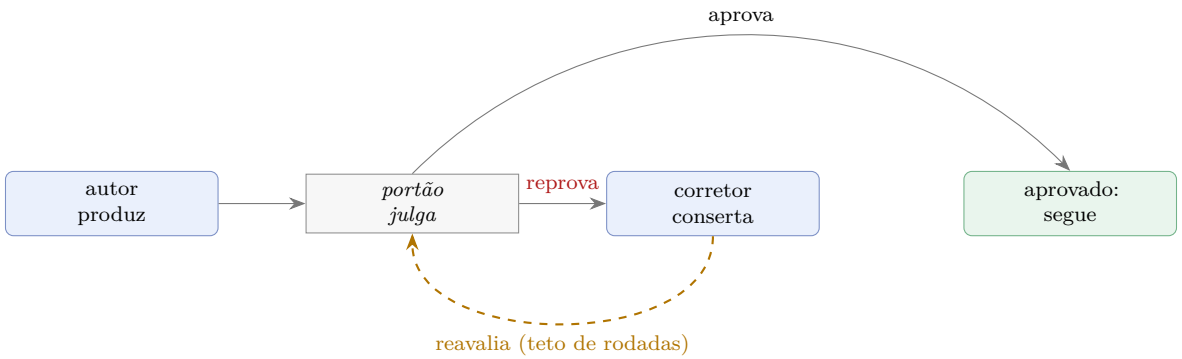


Figura 3: O padrão comum aos seis ciclos: quem julga não conserta, quem conserta não julga

Ciclo	Portão → corretor	O que resolve
Núcleo	micro-gate → 01	*Spoiler*, falta de segredo, escolha sem consequência.
Gate estrutural	05 → 07 revisor	Os 23 critérios de estrutura. O gate escreve o texto de correção pronto para colar.
Spec	currículo + manifesto → 12	Código da BNCC inexistente ou de outro ano; campo do manifesto fora do vocabulário fechado. Conserto mecânico.

QA de design	29 → dono do ticket	Cada achado tem dono declarado; só os endereçados rodam de novo, e o protótipo é refeito depois.
Handoff	telemetria → 33	Contrato de eventos, papéis, canal de sala. O mais barato: a correção vem escrita na mensagem do portão.
Banca	banca → dono do achado	Inclui donos da *etapa anterior*: um erro de jornada detectado em Dev volta para o agente de jornada.

Dois freios. Cada ciclo tem teto próprio, e existe um orçamento global de reexecuções para a execução inteira (`ORCAMENTO_REEXECUCOES`, padrão 8). Estourado qualquer um dos dois, a esteira para de tentar, marca pendência humana e segue até o fim.
O que não volta. Achado cujo dono está na etapa de *Produto* não reabre a spec: reabri-la invalidaria tudo que o Design construiu sobre ela. Vira pendência nomeada no diagnóstico, para decisão humana.

6 Os portões, e o que fazer quando reprovam

Há dois tipos. **Determinísticos** são funções Python: não gastam *token*, não erram e não mudam de opinião. **Avaliadores** são agentes que aplicam rubrica — e aqui vale saber que avaliadores baseados em modelos de linguagem têm vieses documentados de posição, verbosidade e leniência [11], razão pela qual toda regra que *pode* ser mecânica foi tirada deles.

Portão	**Tipo**	**Reprova quando**
Micro-gate (02)	rubrica	O briefing ou o núcleo entrega a virada; a escolha não muda nada.
Gate estrutural (05)	rubrica	Falta mecanismo, caminho sem tratamento, ambiente fora da lista.
Gate final (10)	rubrica	Os três relatórios avaliaram versões diferentes; correção ficou só no relatório.
Currículo BNCC (13)	**código**	Código inexistente, de outro ano, ou sem letramento digital.
Manifesto (14 e 32)	**código**	Campo do bloco 0 ausente ou fora do vocabulário fechado.
QA de design (29)	rubrica	Construção infiel à spec; diagrama ausente ou inválido.
Telemetria (35)	**código**	Evento fora do contrato real da API.
Rastro de lacunas (37)	**código**	Marcadores de lacuna abertos nos documentos.
Banca (38–44)	rubrica	Qualquer um dos critérios bloqueantes das cinco dimensões.
Pendências do PRD (47)	**código**	Marcadores que o redator deixou por falta de informação.

7 Como ler o diagnóstico

O `diagnostico-execucao.md` é o primeiro arquivo a abrir. Ele tem sempre a mesma ordem:

1. **Veredito e contagem** — impedimentos, ressalvas, decisões humanas.
2. **Como cada portão se saiu** — tabela com resultado e rodadas gastas.
3. **O que reprovou, na ordem** — a sequência dos fatos.
4. **O que precisa ser consertado, por dono** — agrupado por responsável.
5. **O que só uma pessoa decide** — o que nenhuma rodada resolve.
6. **O que saiu provisório** — os documentos que dependem de algo reprovado.
7. **Documentos ausentes** — sintoma de agente que não produziu nada.
8. **Por onde começar** — a ação que destrava mais coisa de uma vez.

Regra prática: **impedimento** é o que impede construir; **ressalva** é risco aceito e registrado; **decisão humana** é o que a esteira não tem autoridade para resolver — escolha de produto, licença de imagem, validação pedagógica.

8 O identificador da missão

O código da experiência carrega ano e bimestre:

Código	Significa	Campos que têm de concordar
`exp01ef1b`	1.º ano EF, 1.º bimestre	`series: ["1EF"]` ů `unidade: 1`
`exp05ef3b`	5.º ano EF, 3.º bimestre	`series: ["5EF"]` ů `unidade: 3`
`exp02em2b`	2.º ano EM, 2.º bimestre	`series: ["2EM"]` ů `unidade: 2`

Um portão determinístico confere que `series` e `unidade` concordam com o identificador — divergência trava, porque é ele que nomeia tabela, pasta e *branch*. Missão transversal (sem série) e ferramenta de professor não têm ano nem bimestre: seguem no formato antigo `exp-NN-<slug>`.

9 Configuração

Variável	Padrão	O que faz
`GATE_ESTRUTURAL_MAX_RODADAS`	3	Rodadas do ciclo gate ↔ revisor.
`SPEC_MAX_RODADAS`	2	Ciclo spec ↔ currículo e manifesto.
`QA_DESIGN_MAX_RODADAS`	2	Ciclo QA ↔ retrabalho de design.
`HANDOFF_MAX_RODADAS`	2	Ciclo handoff ↔ telemetria.
`BANCA_MAX_RODADAS`	2	Ciclo banca ↔ dono do achado.
`ORCAMENTO_REEXECUCOES`	8	Teto global de agentes reexecutados.
`ESTEIRA_ESTRITA`	true	Reprovação marca herança degradada e o PRD sai ; desligado, vira ressalva.
`PIPELINE_STAGES`	todas	Permite rodar só uma etapa, sobre um estado salvo.

Qualquer teto em 1 desliga o ciclo correspondente: a esteira avalia e reporta, sem corrigir.

10 Três recomendações de uso

Comece pelo diagnóstico, não pelo PRD.
Ele diz em trinta segundos se vale ler o resto.

Trate `[PROVISÓRIO]` como rascunho, não como entrega.
O marcador existe porque o documento foi construído sobre algo que um portão reprovou.

Rode uma etapa isolada quando estiver iterando.
`PIPELINE_STAGES` sobre um estado salvo custa minutos em vez de uma execução inteira, e a entrada fica fixa entre tentativas.

Referências

[1] CEMRI, M.; PAN, M. Z.; YANG, S. et al. *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025, Datasets and Benchmarks Track. arXiv:2503.13657.

[2] BRASIL. Conselho Nacional de Educação. *Resolução CNE/CEB n.º 1, de 4 de outubro de 2022*: normas sobre Computação na Educação Básica — Complemento à BNCC. Brasília: MEC, 2022.

[3] COWAN, N. The magical number 4 in short-term memory. *Behavioral and Brain Sciences*, v. 24, n. 1, p. 87–114, 2001.

[4] JAROSLAWICZ, D.; WHITING, B.; SHAH, P.; MAAMARI, K. *How Many Instructions Can LLMs Follow at Once?* arXiv:2507.11538, 2025.

[5] JONSSON, A.; SVINGBY, G. The use of scoring rubrics: reliability, validity and educational consequences. *Educational Research Review*, v. 2, n. 2, p. 130–144, 2007.

[6] KNIGHT, J. C.; LEVESON, N. G. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE TSE*, v. SE-12, n. 1, p. 96–109, 1986.

[7] LIU, N. F.; LIN, K.; HEWITT, J. et al. Lost in the middle: how language models use long contexts. *TACL*, v. 12, p. 157–173, 2024.

[8] PARNAS, D. L. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, v. 15, n. 12, p. 1053–1058, 1972.

[9] SWELLER, J. Cognitive load during problem solving: effects on learning. *Cognitive Science*, v. 12, n. 2, p. 257–285, 1988.

[10] TRAN, D.; KIELA, D. *Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets.* arXiv:2604.02460, 2026.

[11] ZHENG, L.; CHIANG, W.-L.; SHENG, Y. et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023. arXiv:2306.05685.